

Smarter Regular Expression Matching

From Theory to Tools

Ilian Kohl

August 8, 2026

Contents

1	Introduction	4
1.1	Motivation	4
1.2	Problem Statement	5
1.3	Contributions	5
1.4	Thesis Outline	6
2	Regular Expressions	7
2.1	Syntax	7
2.2	Semantics	7
2.3	Rust Implementation	8
3	Derivatives and Partial Derivatives	9
3.1	Deciding Membership: An Overview	9
3.2	Brzozowski Derivatives	9
3.3	Antimirov Partial Derivatives	9
3.4	Matcher Selection	9
4	Matching versus Parsing	10
4.1	The Matching Problem	10
4.2	Parse Trees as Matching Evidence	10
4.3	Ambiguity in Regular Expression Parsing	10
4.3.1	Greedy Leftmost Matching	10
4.3.2	The Infinite Empty-Match Chain Problem	10
4.3.3	POSIX Longest-Leftmost Matching	10
5	Derivative-Based Parser	11
5.1	Core Algorithm	11
5.2	Correctness Properties	11
5.3	Bit-Coded Optimization (Theory)	11
5.4	Simplification Rules	11
5.5	Rust Implementation: The <code>inj</code> Version	11
5.5.1	<code>ParseTree</code> and <code>flatten</code>	11
5.5.2	<code>mk_eps</code>	11
5.5.3	<code>inject</code>	11
5.5.4	<code>parse_recursive</code> vs. <code>parse_loop</code>	11
5.6	Rust Implementation: The Bit-Coded Version	11
5.6.1	<code>ARegex</code> - Bit-Annotated Regex	11
5.6.2	<code>internalize/fuse</code>	11

5.6.3	<code>deriv_bc</code>	11
5.6.4	<code>mk_eps_bc</code>	11
5.6.5	<code>decode</code>	11
5.6.6	Heap-Free Parsing	11
6	Partial-Derivative-Based Parser	12
6.1	Extending POSIX Parsing to Partial Derivatives	12
6.2	Bit-Coded Variant	12
6.3	Rust Implementation	12
7	Evaluation	13
7.1	Compared Engines	13
7.1.1	Rust’s <code>regex</code> Crate	13
7.1.2	RE2	13
7.2	Testing Strategy	13
7.3	Correctness Verification	13
7.3.1	Paper Examples Reproduced	13
7.3.2	Cross-Implementation Equivalence	13
7.4	Performance Analysis	13
7.4.1	Stack Depth: Recursive vs. Iterative Construction	13
7.4.2	Heap Allocation: Standard vs. Bit-Coded	13
7.4.3	Derivative-Based vs. Partial-Derivative-Based	13
7.4.4	Pathological Inputs	13
7.4.5	Benchmark Results	13
7.5	Comparison with Existing Engines	13
7.5.1	Performance	13
7.5.2	Standardized Benchmarking via <code>rebar</code>	13
8	Related Work and Discussion	14
8.1	Related Work	14
8.1.1	POSIX-Compliant Engines	14
8.1.2	RE2 and Rust’s <code>regex</code> Crate	14
8.1.3	Other Derivative-Based Approaches	14
8.1.4	Kuklewicz’s Observation on Ordering-Rule Ambiguity	14
8.2	Rust versus Haskell	14
8.2.1	Language Characteristics of the Reference Implementation	14
8.2.2	What the Reference Provides vs. What It Leaves Open	14
8.2.3	Translation Strategy	14
8.2.4	Translation Insights	14
8.3	Mismatch Analysis via Derivative Tracing	14
8.3.1	The Gap	14
8.3.2	Deriving Diagnostics for Free from the Algorithm Itself	14
8.3.3	Design	14
8.3.4	Scope and Limits	14
8.4	Optimization Challenges and Insights	14
9	Conclusion and Future Work	15
9.1	Summary of Contributions	15
9.2	Paper-Suggested Extensions Not Yet Implemented	15

9.2.1	Tagged NFA for Submatching	15
9.2.2	Space-Efficient Parsing	15
9.2.3	DFA/NFA Hybrid Abstraction	15
9.3	Further Performance Optimizations	15
9.4	A Full Mismatch Diagnostics System	15
A	Code Listings	17
B	Full Benchmark Results	18
C	Test Case Catalogue	19
D	CLI and Environment Variable Reference	20

Introduction

1.1 Motivation

Regular expressions are a fundamental formalism for describing regular languages and are among the most widely used abstractions in computer science. They underpin applications including lexical analysis, input validation, log processing, search tools, and network security. Across these applications, the fundamental problem is regular expression matching: given a regular expression and an input string, determine whether the string belongs to the language denoted by the expression. Formally, this is the *membership problem*.

For example, the expression $(ab|a)^*$ matches the input string "aba", as it can be decomposed into "ab" followed by "a", but does not match "abb", since no sequence of subexpressions can produce that input.

For many applications, a Boolean answer is sufficient. An input validator, for instance, may only need to accept or reject a password, URL, or configuration value. Other applications require more: a lexer must determine how an input decomposes into tokens, while capturing groups, schema validation, and information extraction require identifying which parts of the input correspond to different parts of the expression. In these settings, a matcher must recover the *structure* of a match, and the problem shifts from recognition to parsing.

Efficiently solving these problems remains an active area of research: the choice of algorithm directly influences performance, memory consumption, and security, and, once matching becomes parsing, the choice of how a match's structure is recovered directly influences correctness as well.

Existing approaches include backtracking, finite-automata-based techniques such as non-deterministic (NFA) and deterministic (DFA) automata, and derivative-based algorithms. Each approach provides different trade-offs between implementation complexity, memory consumption, and worst-case performance. Backtracking engines are flexible and widely adopted, but may exhibit exponential running time on carefully constructed inputs, leading to regular expression denial-of-service (ReDoS) vulnerabilities [Cox, 2007]. Automata-based approaches provide stronger worst-case guarantees but may require costly preprocessing and can suffer from state-space explosion.

Derivative-based algorithms provide an alternative perspective by transforming the regular expression itself as input symbols are consumed. By maintaining a direct correspondence between an expression and its successive residuals, they offer an elegant foundation for structured regular expression matching.

However, derivative-based techniques are less commonly used in practical regular expression tools than other matching approaches. Bridging this gap between theory and practice motivates the work presented in this thesis.

1.2 Problem Statement

Brzozowski introduced regular expression derivatives as an algebraic characterization of regular expression matching [Brzozowski, 1964]. Antimirov later introduced partial derivatives, which represent sets of residual expressions and provide a construction closely related to nondeterministic automata while reducing redundancy compared with ordinary derivatives [Antimirov, 1996]. Building on these ideas, Sulzmann and Lu presented a derivative-based algorithm for POSIX-correct regular expression parsing [Sulzmann and Lu, 2014].

Although the algorithm is formally developed and accompanied by a Haskell reference implementation, its practical behaviour can be further studied through an implementation in a systems programming language. Rust provides such an environment through its compile-time ownership model, allowing memory allocation, stack usage, and execution time to be studied as direct properties of the implementation.

This thesis investigates the practical implementation of Sulzmann and Lu’s POSIX-correct derivative-based parsing algorithm in Rust. In addition to a direct implementation of the original algorithm, this work explores implementations based on Antimirov’s partial derivatives and the bit-coded optimization proposed by the authors. The resulting implementations are evaluated in terms of performance and resource usage and compared against established regular expression implementations, including Rust’s `regex` crate and Google’s RE2.

Furthermore, the derivative-based representation provides an opportunity beyond conventional matching engines: the intermediate reasoning of the algorithm can be exposed rather than hidden internally. This thesis therefore also investigates how derivative computations can be used as a basis for diagnostic tooling, allowing matches and mismatches to be explained through the transformations performed during the matching process.

1.3 Contributions

This thesis makes the following contributions:

- **A Rust implementation of Sulzmann and Lu’s derivative-based POSIX parser**, including a direct two-pass construction following the original algorithm and a bit-coded single-pass optimization. The implementation additionally explores partial derivatives as an alternative representation.
- **An empirical evaluation** comparing derivative-based and partial-derivative-based implementations with each other and with established regular expression engines, including Rust’s `regex` crate and Google’s RE2.
- **A lightweight derivative-tracing mismatch diagnostics system**, exposing the parser’s intermediate derivative computations and using them to explain successful matches and failures.

1.4 Thesis Outline

The remainder of this thesis is structured as follows. Section 2 introduces the syntax and semantics of regular expressions, together with the Rust representation of their abstract syntax used throughout the rest of this work. Section 3 builds on this representation to introduce Brzozowski’s derivatives and Antimirov’s partial derivatives as algorithms for deciding language membership, along with their Rust implementation – the foundation on which the parsers presented later are built.

Before extending these algorithms to parsing, Section 4 distinguishes matching from parsing and introduces the disambiguation problem that arises once a parse tree, not just a boolean, is required. Section 5 then presents the derivative-based POSIX parser adopted as this thesis’s algorithmic basis, Sulzmann and Lu’s construction [Sulzmann and Lu, 2014], together with its Rust implementation in both a direct ("inj") and a bit-coded form. Section 6 extends this construction from Brzozowski derivatives to Antimirov’s partial derivatives, again in both its direct and bit-coded variants.

With all four parser variants in place, Section 7 evaluates their correctness and performance against each other and against established regular expression engines. Section 8 situates this work relative to related approaches, reflects on translating the underlying algorithm from Haskell to Rust, and discusses the derivative-tracing facility built alongside the parser, including its levels of diagnostic detail and their usefulness. Section 9 closes by summarizing what these four implementations and their evaluation establish, and outlines the extensions – including a fuller mismatch diagnostics system built on this tracing facility, and the algorithmic extensions suggested but not implemented by Sulzmann and Lu left as future work.

Regular Expressions

2.1 Syntax

A regular expression is built from a small set of primitive constructs, combined by three operators. Over an alphabet Σ of symbols, the syntax is given by the following grammar:

$$r, s ::= x \mid \varepsilon \mid \emptyset \mid r + s \mid r \cdot s \mid r^* \quad (x \in \Sigma)$$

Here x denotes a literal symbol, ε the expression matching only the empty word, and $\emptyset$ the expression matching no word at all. The operators $+$, $\cdot$, and $*$ denote alternation, concatenation, and repetition (Kleene star) respectively; concatenation is frequently left implicit, writing rs for $r \cdot s$.

As a running example, consider the problem of recognizing binary strings that end in "01" – a small but non-trivial pattern that touches every construct in the grammar above. Over $\Sigma = \{0, 1\}$, this pattern is expressed as

$$r_{01} = (0 + 1)^* \cdot 0 \cdot 1.$$

Reading this expression against the grammar: $(0 + 1)$ is an alternation of two literals, $(0 + 1)^*$ applies Kleene star to that alternation, and the result is concatenated with the literals 0 and 1 in sequence. This example is used throughout the remainder of this section.

2.2 Semantics

The syntax above describes the *shape* of an expression; the semantics assigns each expression the *language* it denotes. For a regular expression r , write $L(r)$ for the set of words it matches. The semantics is defined by structural recursion on r :

$$\begin{aligned} L(x) &= \{x\} \\ L(\varepsilon) &= \{\varepsilon\} \\ L(\emptyset) &= \{\} \\ L(r + s) &= L(r) \cup L(s) \\ L(r \cdot s) &= \{vw \mid v \in L(r), w \in L(s)\} \\ L(r^*) &= \{\varepsilon\} \cup \{w_1 \cdots w_n \mid w_1, \dots, w_n \in L(r), n \geq 1\} \end{aligned}$$

A word w *matches* r exactly when $w \in L(r)$ – this is the membership problem introduced in Section 1.1, now stated precisely.

Unwinding the semantics for the running example, $L(0+1) = \{0, 1\}$, so $L((0+1)^*)$ is the set of all finite binary strings, including the empty string. Concatenating with $L(0) = \{0\}$ and then $L(1) = \{1\}$ restricts this to

$$L(r_{01}) = \{w \cdot 0 \cdot 1 \mid w \in \{0, 1\}^*\},$$

i.e. exactly the binary strings ending in "01" – "01", "001", "1101", and so on, but not "010" or the empty string. This last point is worth making explicit: r_{01} is *not* nullable – $\varepsilon \notin L(r_{01})$, since every word in $L(r_{01})$ must end in the literal 1. Nullability, i.e. whether $\varepsilon \in L(r)$, is a property later chapters compute directly on the syntax of r rather than by reasoning about $L(r)$; the two are guaranteed to agree by the semantics defined here.

2.3 Rust Implementation

The grammar above is represented directly as a Rust enum, mirroring each syntactic construct one to one:

```
#[derive(Debug, Clone, PartialEq, Eq, Hash)]
pub enum Regex {
    Phi,
    Eps,
    Lit(char),
    Seq(Box<Regex>, Box<Regex>),
    Alt(Box<Regex>, Box<Regex>),
    Star(Box<Regex>),
}
```

This one-to-one correspondence is not incidental. Rust’s enums are algebraic data types: each variant corresponds to exactly one production of the grammar, and Rust’s `match` requires every variant to be handled, rejecting the program at compile time otherwise. This gives the same guarantee an inductive definition of r in Section 2.1 carries on paper – that a function defined by cases on r is defined for every possible r – as a property the compiler checks, rather than one that must be checked by inspection. This is a structural fit shared with other languages that support algebraic data types, including Haskell itself; Rust’s specific contribution is not this correspondence but what it costs to maintain, which is not always nothing. `Phi` and `Eps` carry no data, matching $L(\emptyset) = \{\}$ and $L(\varepsilon) = \{\varepsilon\}$ directly, and `Lit` carries the literal symbol x – these three variants transcribe directly with no adjustment. The recursive constructs – `Seq`, `Alt`, `Star` – do not: unlike Haskell’s `data R = ... | Seq R R | ...`, whose recursive constructors are represented indirectly by the runtime with no annotation required, Rust must know the exact size of a type at compile time, and a `Regex` containing a `Regex` directly would have unbounded size. `Box<Regex>` breaks the recursion by storing the subexpression on the heap and holding only a pointer inline – the first of several places in this thesis where a construct free in the Haskell reference requires an explicit, visible workaround in Rust, revisited more generally in Section 8.2.

Derivatives and Partial Derivatives

3.1 Deciding Membership: An Overview

3.2 Brzozowski Derivatives

3.3 Antimirov Partial Derivatives

3.4 Matcher Selection

Matching versus Parsing

4.1 The Matching Problem

4.2 Parse Trees as Matching Evidence

4.3 Ambiguity in Regular Expression Parsing

4.3.1 Greedy Leftmost Matching

4.3.2 The Infinite Empty-Match Chain Problem

4.3.3 POSIX Longest-Leftmost Matching

Derivative-Based Parser

5.1 Core Algorithm

5.2 Correctness Properties

5.3 Bit-Coded Optimization (Theory)

5.4 Simplification Rules

5.5 Rust Implementation: The inj Version

5.5.1 ParseTree and flatten

5.5.2 mk_eps

5.5.3 inject

5.5.4 parse_recursive vs. parse_loop

5.6 Rust Implementation: The Bit-Coded Version

5.6.1 ARegex - Bit-Annotated Regex

5.6.2 internalize/fuse

5.6.3 deriv_bc

5.6.4 mk_eps_bc

5.6.5 decode

5.6.6 Heap-Free Parsing

Partial-Derivative-Based Parser

6.1 Extending POSIX Parsing to Partial Derivatives

6.2 Bit-Coded Variant

6.3 Rust Implementation

Evaluation

7.1 Compared Engines

7.1.1 Rust's regex Crate

7.1.2 RE2

7.2 Testing Strategy

7.3 Correctness Verification

7.3.1 Paper Examples Reproduced

7.3.2 Cross-Implementation Equivalence

7.4 Performance Analysis

7.4.1 Stack Depth: Recursive vs. Iterative Construction

7.4.2 Heap Allocation: Standard vs. Bit-Coded

7.4.3 Derivative-Based vs. Partial-Derivative-Based

7.4.4 Pathological Inputs

7.4.5 Benchmark Results

7.5 Comparison with Existing Engines

7.5.1 Performance

7.5.2 Standardized Benchmarking via rebar

Related Work and Discussion

8.1 Related Work

8.1.1 POSIX-Compliant Engines

8.1.2 RE2 and Rust's regex Crate

8.1.3 Other Derivative-Based Approaches

8.1.4 Kuklewicz's Observation on Ordering-Rule Ambiguity

8.2 Rust versus Haskell

8.2.1 Language Characteristics of the Reference Implementation

8.2.2 What the Reference Provides vs. What It Leaves Open

8.2.3 Translation Strategy

8.2.4 Translation Insights

8.3 Mismatch Analysis via Derivative Tracing

8.3.1 The Gap

8.3.2 Deriving Diagnostics for Free from the Algorithm Itself

8.3.3 Design

8.3.4 Scope and Limits

8.4 Optimization Challenges and Insights

Conclusion and Future Work

9.1 Summary of Contributions

9.2 Paper-Suggested Extensions Not Yet Implemented

9.2.1 Tagged NFA for Submatching

9.2.2 Space-Efficient Parsing

9.2.3 DFA/NFA Hybrid Abstraction

9.3 Further Performance Optimizations

9.4 A Full Mismatch Diagnostics System

Bibliography

- Valentin Antimirov. Partial derivatives of regular expressions and finite automaton constructions. *Theoretical Computer Science*, 155(2):291–319, 1996. doi: 10.1016/0304-3975(95)00182-4.
- Janusz A. Brzozowski. Derivatives of regular expressions. *Journal of the ACM*, 11(4):481–494, 1964. doi: 10.1145/321239.321249.
- Russ Cox. Regular expression matching can be simple and fast (but is slow in Java, Perl, PHP, Python, Ruby, ...). <https://swtch.com/~rsc/regexp/regexp1.html>, 2007. Accessed: 2026-08-05.
- Martin Sulzmann and Kenny Zhuo Ming Lu. Posix regular expression parsing with derivatives. In *Functional and Logic Programming (FLOPS 2014)*, volume 8475. Springer, 2014.

Code Listings

Full Benchmark Results

Test Case Catalogue

CLI and Environment Variable Reference